

Nathaniel Abera

August 20, 2026

Software Security and Testing

A1.

Hardcoded Secrets – A1a

Plaintext Passwords – A1b

A2.

Secure Env Variables – A2a

Password Hashing – A2b

A3.

General Commented Out – A3

B1.

Missing API Auth – B1a

```
206 def require_api_key():
207     """
208     TODO (Section B): Implement API Key Authentication
209
210     This function should:
211     1. Extract the API key from the Authorization header
212        | - Expected format: "Bearer <api_key>"
213     2. Validate the API key against USERS_DB
214     3. Return the username if valid, None if invalid
215
216     SECURITY NOTE: API keys should be transmitted in headers, NOT URL parameters.
217     See CWE-598 (Use of GET Request Method With Sensitive Query Strings).
218     """
219     # INSECURE: Currently returns None (no authentication)
220     # TODO: Implement proper API key validation
221     return None
```

Broken Auth – B1b

```
248 def require_role(username, required_role):
249     """
250     TODO (Section B): Implement Role-Based Authorization (Least Privilege)
251
252     This function should:
253     1. Look up the user's role from USERS_DB
254     2. Check if the user's role meets the required permission level
255     3. Return True if authorized, False otherwise
256
257     Role hierarchy (suggestion):
258     - admin: can access everything
259     - user: can access user-level endpoints
260     - guest: limited read-only access
261     """
262     # INSECURE: Currently always returns True (no authorization)
263     # TODO: Implement proper role checking
264     return True
```

B2.

API Key Validation – B2a

```
# REMEDIATED (B2) - original insecure code disabled, do not re-enable:
# return None

auth_header = request.headers.get('Authorization', '')
if not auth_header.startswith('Bearer '):
    return None
supplied_key = auth_header.split(' ', 1)[1]
for username, record in USERS_DB.items():
    if secrets.compare_digest(record['api_key'], supplied_key):
        return username
return None
```

Secure Role Authorization – B2b

```
# REMEDIATED (B2) - original insecure code disabled, do not re-enable:
# return True

role_levels = {"guest": 1, "user": 2, "admin": 3}
record = USERS_DB.get(username)
if not record:
    return False
return role_levels.get(record['role'], 0) >= role_levels.get(required_role, 0)
```

B3.

API Commented Out – B3

```
227 def require_api_key():
228     """
229     TODO (Section B): Implement API Key Authentication
230
231     This function should:
232     1. Extract the API key from the Authorization header
233     - Expected format: "Bearer <api_key>"
234     2. Validate the API key against USERS_DB
235     3. Return the username if valid, None if invalid
236
237     SECURITY NOTE: API keys should be transmitted in headers, NOT URL parameters.
238     See CWE-598 (Use of GET Request Method With Sensitive Query Strings).
239     """
240     # REMEDIATED (B2) - original insecure code disabled, do not re-enable:
241     # return None
242
243 277 def require_role(username, required_role):
278     """
279     TODO (Section B): Implement Role-Based Authorization (Least Privilege)
280
281     This function should:
282     1. Look up the user's role from USERS_DB
283     2. Check if the user's role meets the required permission level
284     3. Return True if authorized, False otherwise
285
286     Role hierarchy (suggestion):
287     - admin: can access everything
288     - user: can access user-level endpoints
289     - guest: limited read-only access
290
291     """
292     # REMEDIATED (B2) - original insecure code disabled, do not re-enable:
293     # return True
```

C1.

Naming Conventions – C1

```
import pytest
from app_solution import app, USERS_DB

# =====
# GENERAL SECURITY VULNERABILITY TESTS (Section A remediations)
# =====

def test_general_vuln_1_hardcoded_secrets_remediated():
    assert app.secret_key != 'supersecretkeyforflasksessions'
    print("PASS: Secret key is no longer hardcoded")

def test_general_vuln_2_plaintext_passwords_remediated():
    for username in USERS_DB:
        assert 'password' not in USERS_DB[username]
        assert 'password_hash' in USERS_DB[username]
    print("PASS: All passwords are now stored as hashes")
```

C2.

Statement Assertions – C2

```
def test_general_vuln_1_hardcoded_secrets_remediated():
    assert app.secret_key != 'supersecretkeyforflasksessions'
    print("PASS: Secret key is no longer hardcoded")

def test_general_vuln_2_plaintext_passwords_remediated():
    for username in USERS_DB:
        assert 'password' not in USERS_DB[username]
        assert 'password_hash' in USERS_DB[username]
    print("PASS: All passwords are now stored as hashes")
```

C3.

Successful Test Execution – C3

```
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter\
.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter
collected 4 items

test_run.py::test_general_vuln_1_hardcoded_secrets_remediated PASS: Secret key is no longer hardcoded
PASSED
test_run.py::test_general_vuln_2_plaintext_passwords_remediated PASS: All passwords are now stored as hashes
PASSED
test_run.py::test_api_vuln_1_authentication_required PASS: Unauthenticated request was denied with 401 Unauthorized
PASSED
test_run.py::test_api_vuln_2_authorization_enforced PASS: Non-admin user was denied admin access with 403 Forbidden
PASSED

===== 4 passed in 0.46s =====
```

C4.

Descriptive Console Output – C4

```
test_run.py::test_general_vuln_1_hardcoded_secrets_remediated PASS: Secret key is no longer hardcoded
PASSED
test_run.py::test_general_vuln_2_plaintext_passwords_remediated PASS: All passwords are now stored as hashes
PASSED
test_run.py::test_api_vuln_1_authentication_required PASS: Unauthenticated request was denied with 401 Unauthorized
PASSED
test_run.py::test_api_vuln_2_authorization_enforced PASS: Non-admin user was denied admin access with 403 Forbidden
PASSED
```

C5.

Hardcoded Secrets Pass – C5a

```
(.venv) PS C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter> pytest test_run.py::test_general_vuln_1_hardcoded_secrets_remediated -v -s
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter\
.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter
collected 1 item

test_run.py::test_general_vuln_1_hardcoded_secrets_remediated PASS: Secret key is no longer hardcoded
PASSED

===== 1 passed in 0.43s =====
```

Plaintext Passwords Pass – C5b

```
(.venv) PS C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter> pytest test_run.py::test_general_vuln_2_plaintext_passwords_remediated -v -s
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter\
.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter
collected 1 item

test_run.py::test_general_vuln_2_plaintext_passwords_remediated PASS: All passwords are now stored as hashes
PASSED

===== 1 passed in 0.42s =====
```

API Auth Pass (401 Unauthorized) – C5c

```
(.venv) PS C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter> pytest test_run.py::test_api_vuln_1_authentication_required -v -s
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter\
.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter
collected 1 item

test_run.py::test_api_vuln_1_authentication_required PASS: Unauthenticated request was denied with 401 Unauthorized
PASSED

===== 1 passed in 0.44s =====
```

API Auth Pass (403 Forbidden) – C5d

```
(.venv) PS C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter> pytest test_run.py::test_api_vuln_2_authorization_enforced -v -s
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter\
.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\D385 Task 2 Files - Nathaniel Abera\d385_software_security_and_testing-task-2-student-starter
collected 1 item

test_run.py::test_api_vuln_2_authorization_enforced PASS: Non-admin user was denied admin access with 403 Forbidden
PASSED

===== 1 passed in 0.43s =====
```

D1. The first mitigation strategy that I implemented was externalizing the credential management and secrets that were hardcoded, as well as plaintext password storage identified in the flake8 report. I moved the Flask session key, API key, and database credentials outside of the source code and instead placed them in environment variables that are read at runtime, with a randomly generated fallback session key if no value is supplied. The credentials are no longer committed to version control, and they can now be rotated without modifying the application itself. The user database now stores PBKDF2-HMAC-SHA256 hashes generated with a salt and 100,000 iterations rather than the original passwords. As recommended by NIST, a salted iterated key

function means a database compromise does not immediately expose usable credentials (National Institute of Standards and Technology [NIST], 2025). These fixes align with the same principle and show that sensitive values should never be able to be directly readable within an application.

The second mitigation strategy that I implemented was enforcing authentication with least-privilege authorization, which addressed the missing API authentication and broken authorization vulnerabilities. Previously the `require_api_key` would return “None” on every call so that means there was no endpoint and it wasn’t able to authenticate anything. But now it will extract the Bearer API token from the Authorization header and it will validate it against the stored API keys by using a comparison which would return the associated username only on a match. As for the `require_role` function it previously would return “True” on every call which granted every user admin access. But now it will not do that and it will compare the user that’s requesting it against a ranked permission list and it will deny a request if the user requesting it does not have a specific role. Now both functions fail closed which means that credentials that are not authorized or unrecognized will get denied rather than getting access for both the administrative endpoints and the rental ones.

D2. The first technique that I used was boundary and type checking, which confirms a value is the expected data type and falls within an acceptable range before it’s used. For example, in the `create_rental` the equipment rate was being multiplied by the days value with no checks so a non-numeric value would just come back as a `TypeError` and the total would just produce a negative amount. By verifying the days is an integer between a minimum positive numeric value such as one and a defined maximum it would reject both of them before they ever reached the calculation.

The second technique that I used was adding an allowlist for validation. This would compare inputs against a set of defined acceptable values and if anything was not within this list it would be rejected. This is stronger than a blocklist because a blocklist only stops the specific patterns someone thought to include, while an allowlist denies everything that was not explicitly permitted. In `search_equipment`, for example, the raw query parameter is reflected back in the

response, which allows for the script content to reach the client. Restricting the term to a defined character set and escaping it before an output would close that gap.

D3. The first improvement I made to exception handling was adding a generic error message within the client when logging exception details on the server because the current handler, `get_user_data`, would just return `str(e)` in the response body and this isn't good because it exposes internal details like file paths, query text, and database structures to anyone who triggers that error. This is CWE-209, information exposure through an error message. Replacing that response with a fixed message reading "An internal error occurred" removes the disclosure, while at the same time writing the full traceback to a log on the server preserves exactly what a developer needs. This improves security by denying an attacker the reconnaissance value of error output, and it improves reliability because failures are still captured in a durable log instead of it being lost once a respond is sent.

The second improvement I did for exception handling was instead of using a bare `except` Exception block I would use specific exception types. The current handler catches every exception the same exact way, so a database connection failure, a malformed query, and an unrelated programming error all will produce the same identical 500 response. By adding a specific exception rule it will catch things like `sqlite3.Error` separately from other errors that may appear like `ValueError`. This improves security because an overly broad handler will silently absorb a failed validation or even an authorization check and it will let an execution continue even during an unintended state. This improves reliability because it allows for genuine defects to surface themselves rather than it being a hidden genuine server error which makes it easier to give a proper diagnostic.

D4. The encryption method that was used to secure the application traffic was the Transport Layer Security, also known as TLS. This uses a handshake to authenticate a certificate with the server, and it gives a shared session key that allows for all requests and response data to be encrypted when it's being transmitted. This is very important because it prevents traffic from

being intercepted when it's sent through or out of a network. The Bearer tokens from Section B are a good example of this, as they travel with the Authorization header inside of every request and if there wasn't a Transport Layer Security (TLS) layer implemented then they would be readable by anyone on the network path. Transport Layer Security (TLS) provides these core principles: integrity, confidentiality, and server authentication (OWASP Foundation, n.d.).

References

National Institute of Standards and Technology. (2025). *Digital identity guidelines: Authentication and authenticator management* (NIST Special Publication 800-63B-4). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-63b-4>

OWASP Foundation. (n.d.). *Transport layer security cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html